

nlmixr2: survival models

(and other odds and ends)

R/Pharma 2026

Justin Wilkins, PhD

On behalf of the **nlmixr** development team:

Matt Fidler, Bill Denney, John Harrold, Richard Hooijmaijers,
Mutaz Jaber, Anne Keunecke, Theo Papathanasiou, Rik
Schoemaker, Max Taubert, Mirjam Trame, Justin Wilkins,
Brendan Bender

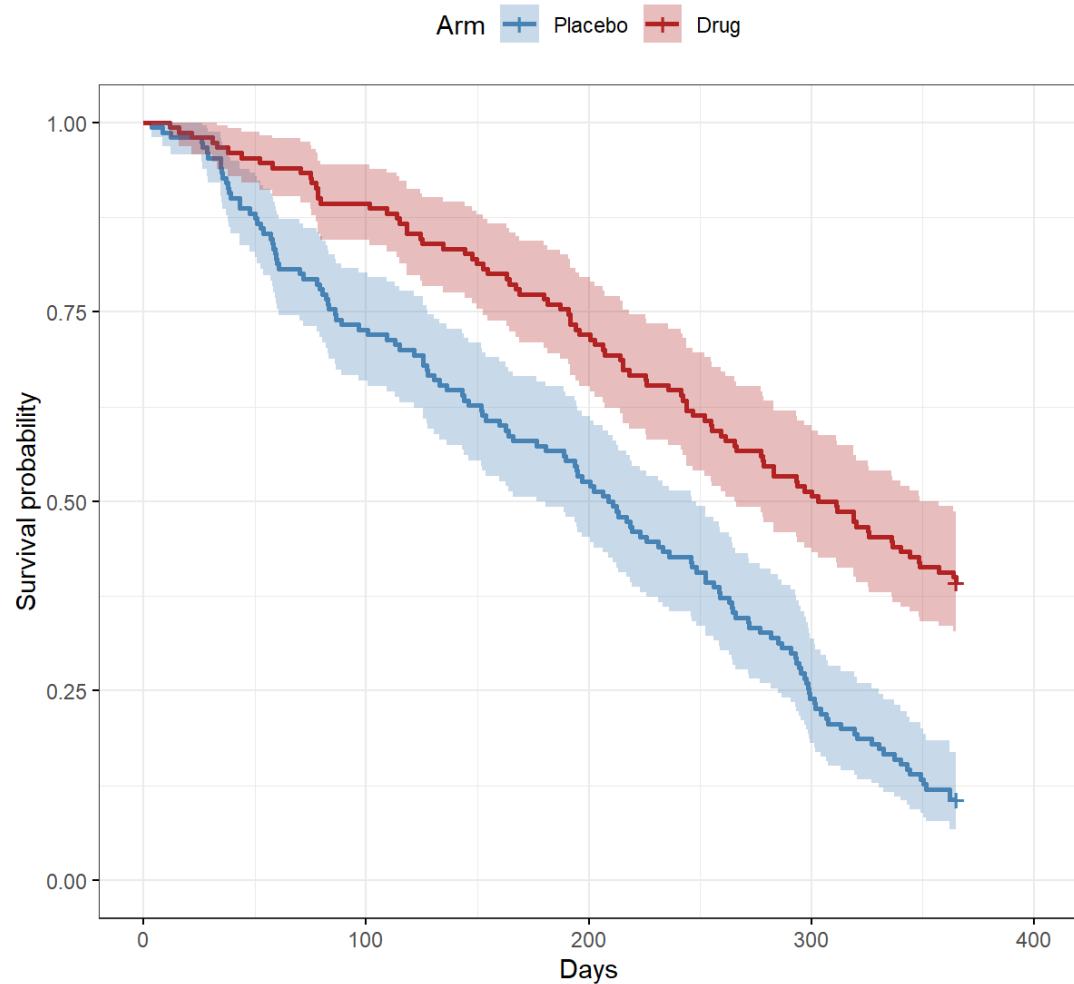


Survival / time to event (TTE) models

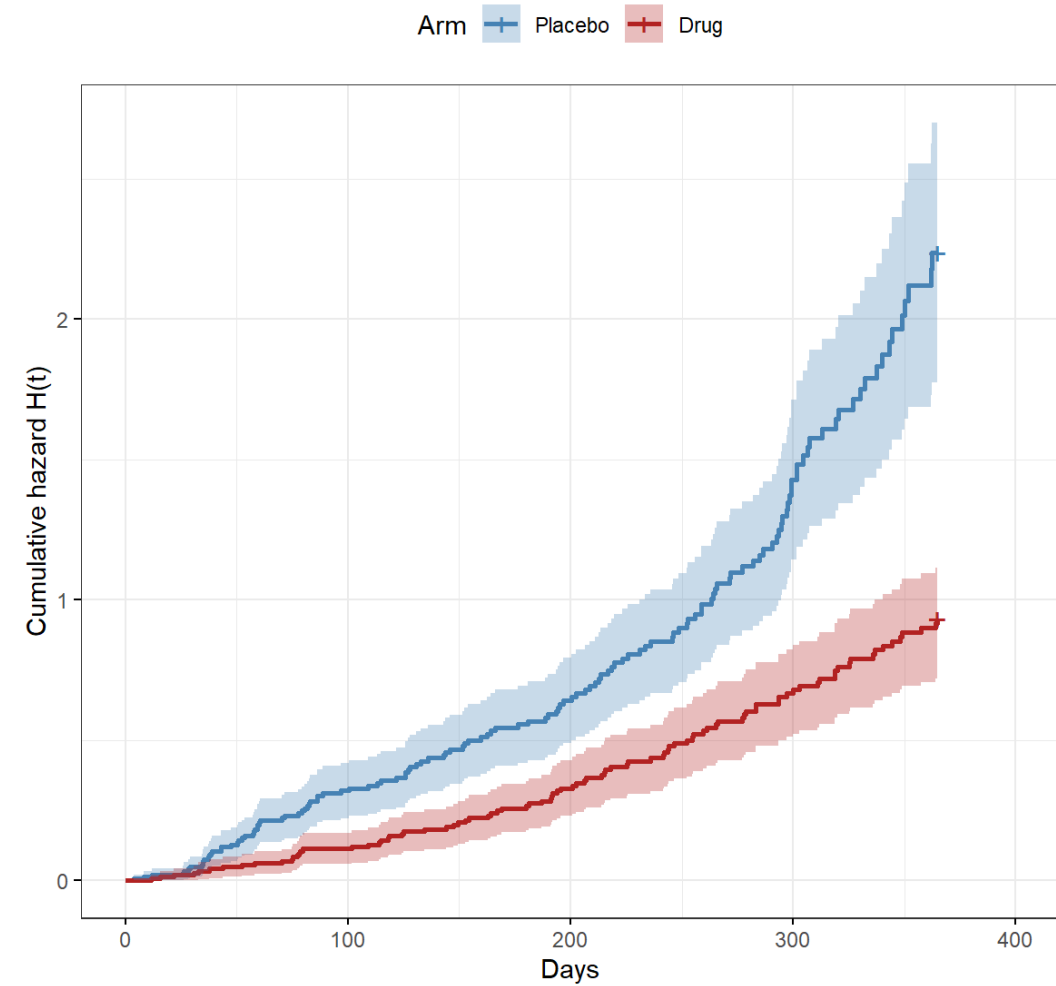
- Time-to-event (TTE) analysis models the time until a discrete event occurs
 - Tumor progression, an adverse event, dropout, relapse, device failure, death, etc
- The endpoint is not only whether the event occurred, but also how long the subject remained event-free
 - A subject who is followed for 24 weeks without any event is therefore informative even though an event was not observed

Consider a straightforward example...

Kaplan-Meier curves by treatment arm (simulated Gompertz data)



Cumulative hazard by treatment arm



Let's try an exponential model first, just to see

```
tte_exponential_model <- function() {  
  ini({  
    # Baseline log hazard for the control group  
    log_h0 <- log(0.004)  
    # Log hazard ratio for treatment versus control  
    trt_log_hr <- -0.1  
  })  
  model({  
    # Treatment multiplies the baseline hazard on the hazard-ratio scale  
    log_h <- log_h0 + test_log_hr * test_trt  
    h <- exp(log_h)  
    # Exponential cumulative hazard  
    H <- h * time  
    # Event rows contribute log(h) - H; censored rows contribute -H  
    tte_ll <- event * log_h - H  
    ll(tte) ~ tte_ll  
  })  
}  
  
# Use bobyqa for this custom likelihood to avoid relying on symbolic gradients  
fit_exp <- nlmixr(exp_model, tte_sim, est = "bobyqa",  
  control = bobyqaControl(print = 0))
```

TTE models are supported in nlmixr2 using custom likelihoods – `ll()`

$$h(t) = \lambda$$

$$H(t) = \lambda t$$

nlmixr2 fit output

```
> fit_exp
```

```
— nlmixr2 log-likelihood bobyqa —
```

	OBJF	AIC	BIC	Log-likelihood	Condition#(Cov)	Condition#(Cor)
lPop	2457.356	3012.719	3020.127	-1504.359	6.027417	4.503912

```
— Time (sec fit_exp$time): —
```

	setup	optimize	covariance	preprocess	postprocess	table	compress	other
elapsed	0.2636977	0.36145	0.0001606	0.08	0	0.03	0	0.5946917

```
— (fit_exp$parFixed or fit_exp$parFixedDf): —
```

	Parameter	Est.	SE	%RSE	Back-transformed(95%CI)
log_h0	log baseline hazard (/day)	-5.42	0.0866	1.60	-5.42 (-5.59, -5.25)
test_log_hr	log hazard ratio, drug vs. placebo	-0.668	0.136	20.4	-0.668 (-0.935, -0.402)

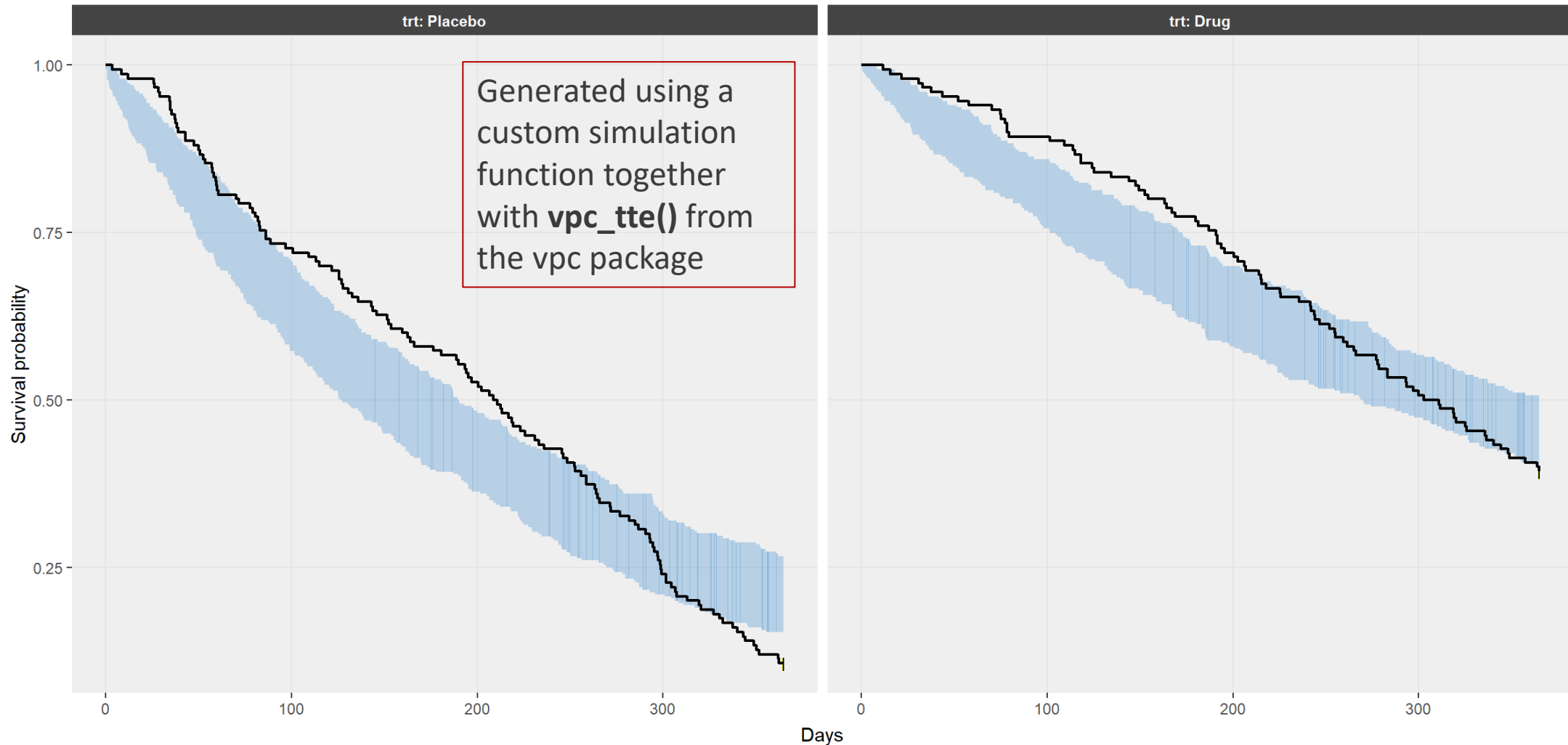
```
Covariance Type (fit_exp$covMethod): r
```

```
Fixed parameter correlations in fit_exp$cor
```

```
Censoring (fit_exp$censInformation): No censoring
```

VPC suggests there's room for improvement

Exponential TTE VPC by treatment arm



Now for a Gompertz model (non-proportional hazard)

```
gompertz_model <- function() {  
  ini({  
    log_alpha <- log(0.003) # log hazard at t = 0 (/day)  
    log_gamma <- log(0.003) # log growth rate; estimating on the log scale keeps gamma >  
0  
    test_log_hr <- -0.5 # informed by KM: ~40-50% separation visible  
  })  
  model({  
    alpha <- exp(log_alpha)  
    gamma <- exp(log_gamma)  
    h <- alpha * exp(gamma * time) * exp(test_log_hr * test_trt)  
    H <- (alpha / gamma) * (exp(gamma * time) - 1) * exp(test_log_hr * test_trt)  
    tte_ll <- event * log(h) - H  
    ll(tte) ~ tte_ll  
  })  
}  
  
# Fit the Gompertz model to the same TTE data  
fit_gompertz <- nlmixr(gompertz_model, tte_sim, est = "bobyqa",  
  control = bobyqaControl(print = 0))
```

$$h(t) = \frac{\gamma}{\lambda} \cdot \left(\frac{t}{\lambda}\right)^{\gamma-1}$$

$$H(t) = \left(\frac{t}{\lambda}\right)^{\gamma}$$

This looks a bit better...

```
> fit_gompertz
```

```
— nlmixr2 log-likelihood bobyqa —
```

	OBJF	AIC	BIC	Log-likelihood	Condition#(Cov)	Condition#(Cor)
lPop	2404.279	2961.642	2972.753	-1477.821	17.93195	17.72155

```
— Time (sec fit_gompertz$time): —
```

	setup	optimize	covariance	preprocess	postprocess	table	compress	other
elapsed	0.2824439	0.2805004	5.7e-06	0.08	0	0.02	0	0.22705

```
— (fit_gompertz$parFixed or fit_gompertz$parFixedDf): —
```

	Parameter	Est.	SE	%RSE
log_alpha	log hazard at t = 0 (/day)	-6.17	0.147	2.38
log_gamma	log growth rate; estimating on the log scale keeps gamma > 0	-5.32	0.137	2.57
test_log_hr	informed by KM: ~40-50% separation visible	-0.799	0.137	17.1
Back-transformed(95%CI)				
log_alpha		0.00210	(0.00157, 0.00280)	
log_gamma		0.00487	(0.00372, 0.00637)	
test_log_hr		-0.799	(-1.07, -0.531)	

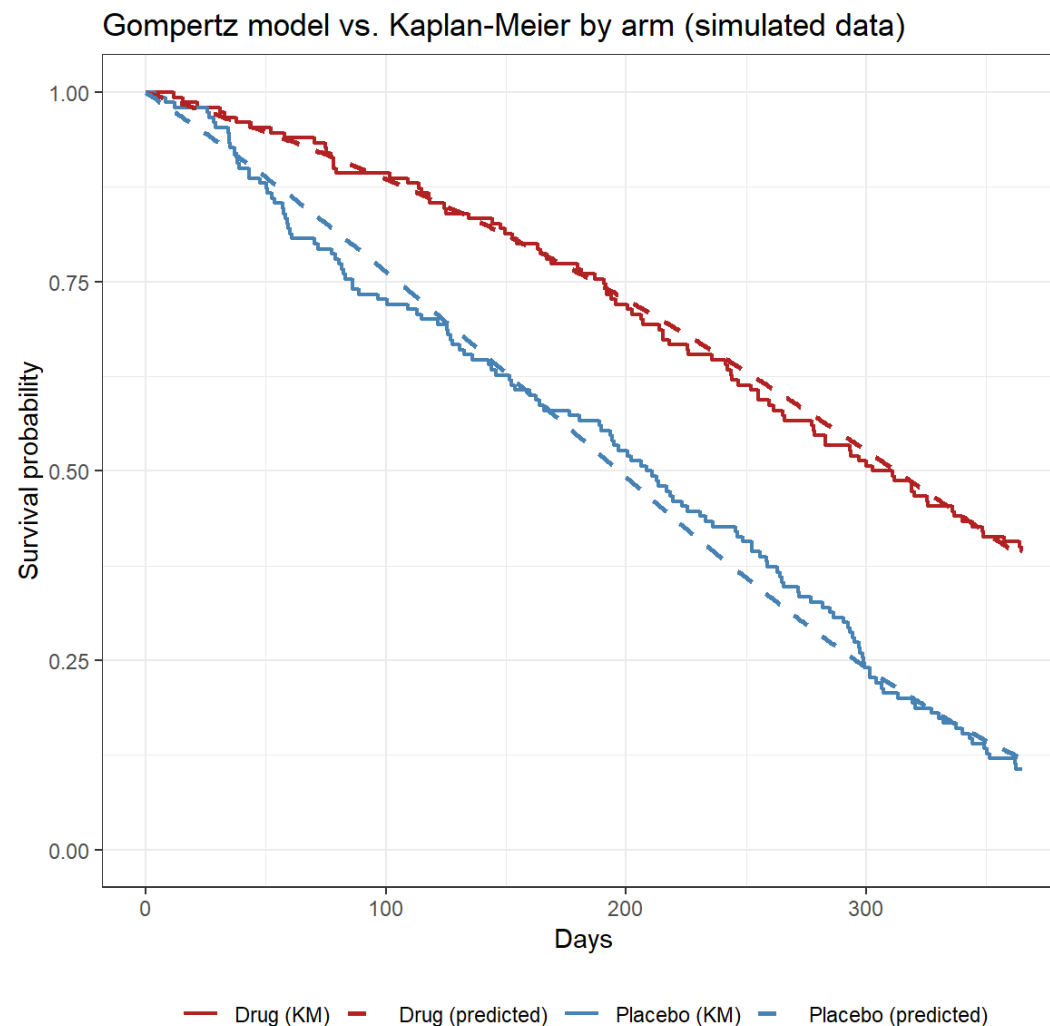
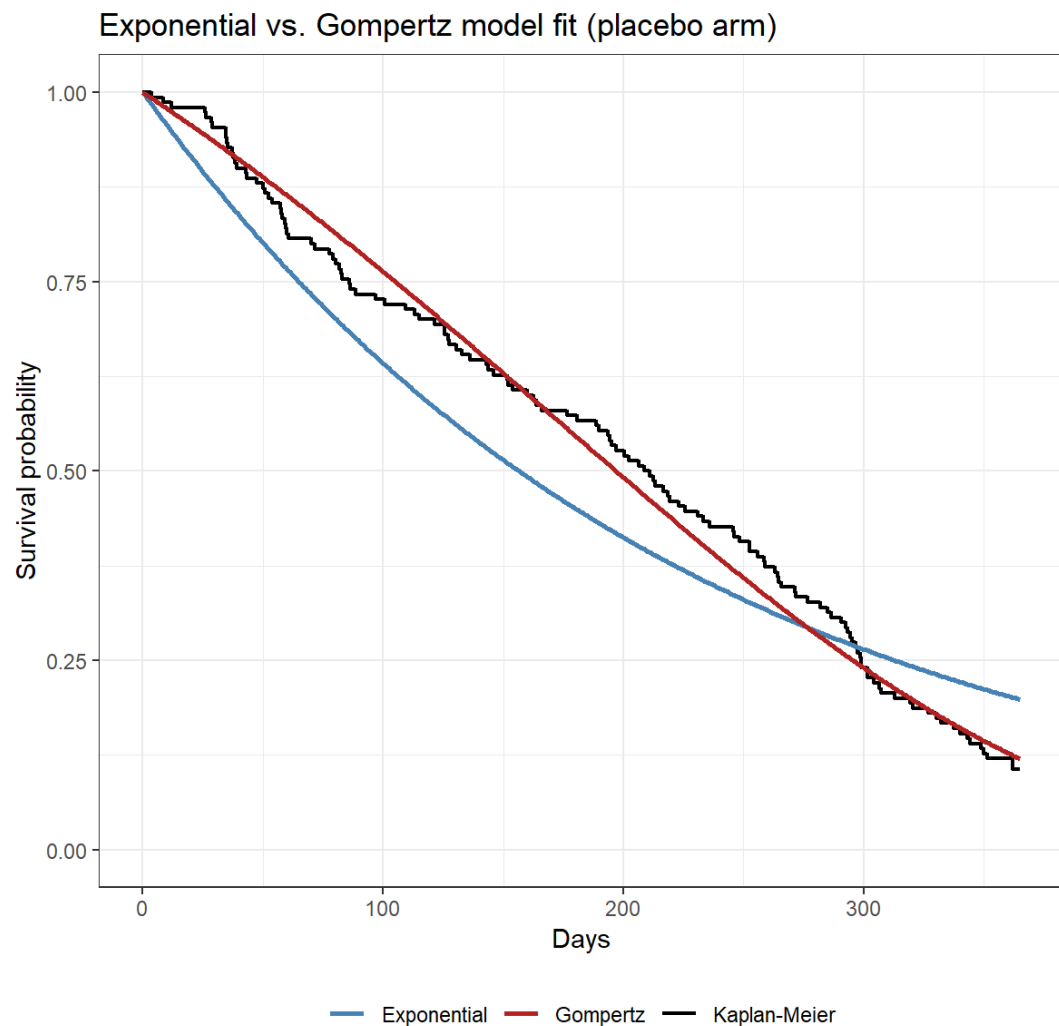
```
Covariance Type (fit_gompertz$covMethod): r
```

```
Some strong fixed parameter correlations exist (fit_gompertz$cor) :
```

cor:log_gamma,log_alpha	cor:test_log_hr,log_alpha	cor:test_log_hr,log_gamma
-0.810	-0.258	-0.138

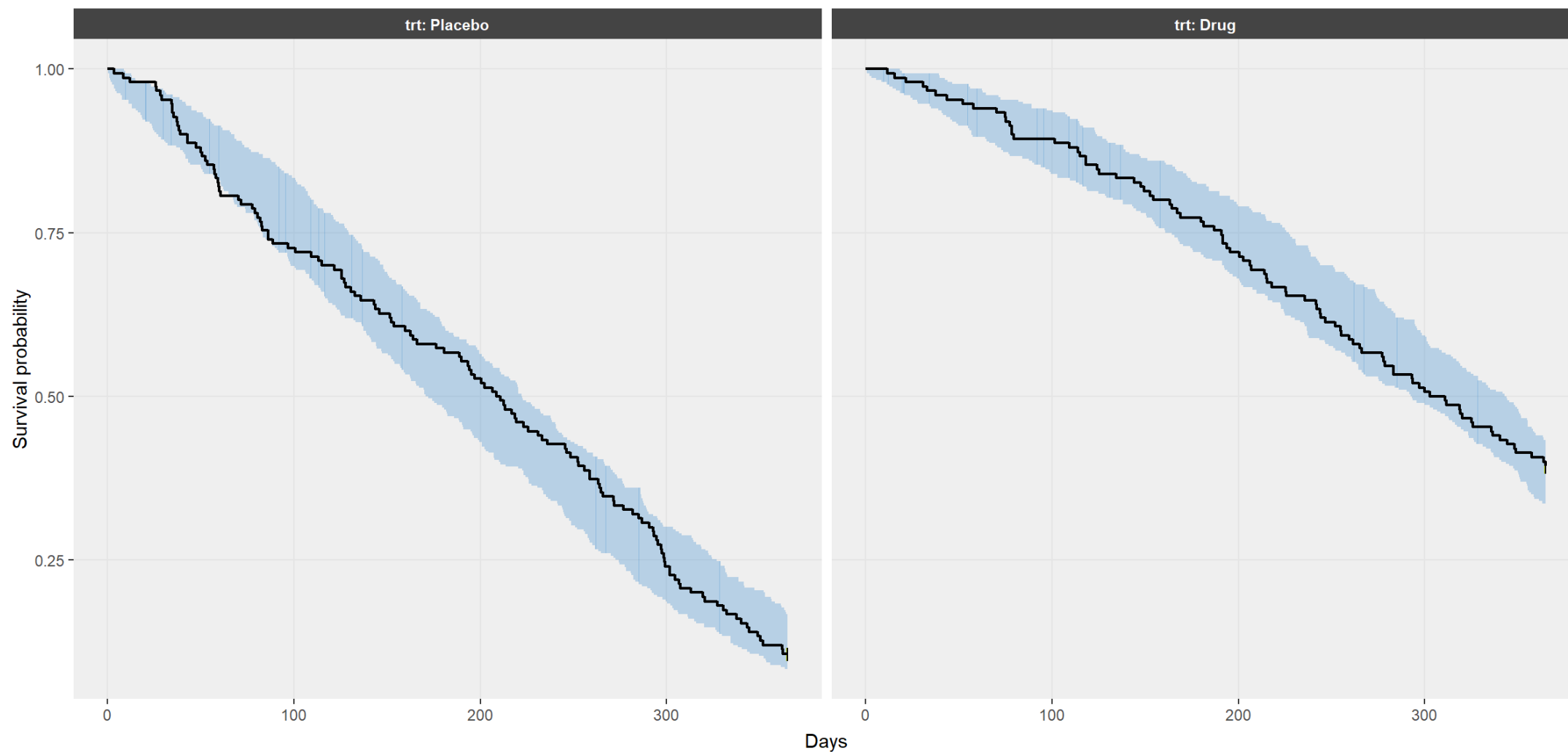
```
Censoring (fit_gompertz$censInformation): No censoring
```

How did we do?



The VPC looks better!

Gompertz TTE VPC by treatment arm



Advanced TTE modeling

- The example we show here could just as easily be performed using the **survreg** package in R – these models are quite simple
- Where it starts becoming really useful is with repeated time-to-event (RTTE) models, or with joint models (e.g. TGI-OS)
- Code examples are provided

Models for discrete data

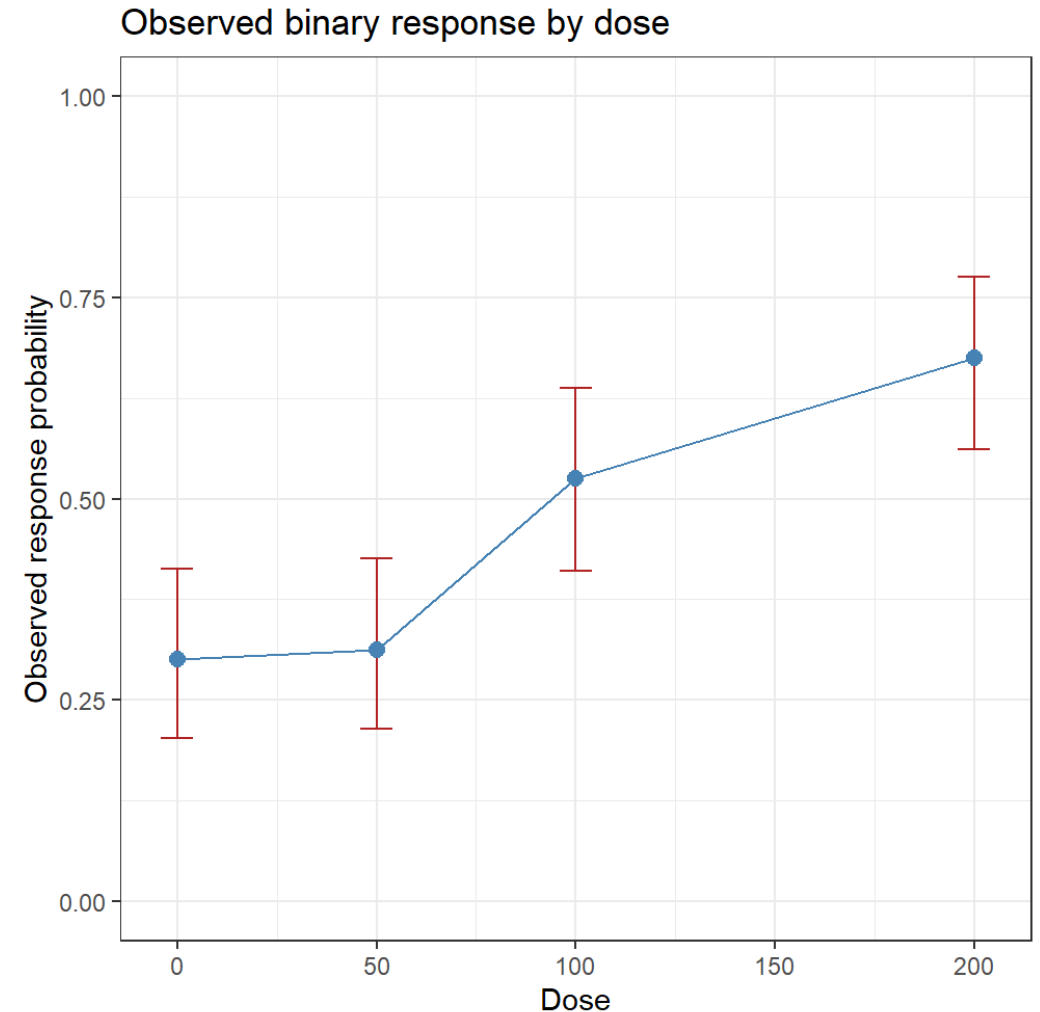
- Discrete data can be:
 - Binary (0 or 1, yes or no)
 - Categorical (ordered or unordered)
 - Counts
- These kinds of data cannot easily be approximated using continuous-data models

Models for binary data

- We typically use logistic regression for binary data

$$\text{logit}(p_i) = \log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 \cdot x_{1,i} + \dots + \beta_n \cdot x_{n,i}$$

```
logistic_binary_model <- function() {  
  ini({  
    # Log odds of response at dose 0  
    logit_p0 <- -1  
    # Change in log odds for each one-unit increase in dose  
    beta_dose <- 0.008  
  })  
  model({  
    # Linear predictor on the log-odds scale  
    lp <- logit_p0 + beta_dose * DOSE  
    # Inverse-logit transformation to obtain a probability  
    p <- expit(lp)  
    # A binary response is a binomial endpoint with one trial  
    one <- 1  
    RESPONSE ~ dbinom(one, p)  
  })  
  
  # Fit the logistic regression model  
  fit_logistic_binary <- nlmixr(  
    logistic_binary_model, binary_data, est = "nlm",  
    control = nlmControl(print = 0))  
}
```



Model fit

This suggests the odds ratio per 100 unit increase in dose is

$$\exp(100 \cdot 0.00733) = 2.08$$

```
> fit_logistic_binary
```

```
— nlmixr2 log-likelihood nlm —
```

	OBJF	AIC	BIC	Log-likelihood	Condition#(Cov)	Condition#(Cor)
lPop	-177.0573	415.0633	422.6	-205.5317	34063.24	8.09268

```
— Time (sec fit_logistic_binary$time): —
```

	setup	optimize	covariance	preprocess	postprocess	table	compress	other
elapsed	0.2922387	0.430681	5.2e-06	0.08	0	0.03	0	0.1770751

```
— (fit_logistic_binary$parFixed or fit_logistic_binary$parFixedDf): —
```

	Est.	SE	%RSE	Back-transformed(95%CI)
logit_p0	-1.03	0.187	18.2	-1.03 (-1.40, -0.663)
beta_dose	0.00733	0.00162	22.1	0.00733 (0.00416, 0.0105)

```
Covariance Type (fit_logistic_binary$covMethod): r (nlm)
```

```
Some strong fixed parameter correlations exist (fit_logistic_binary$cor) :
```

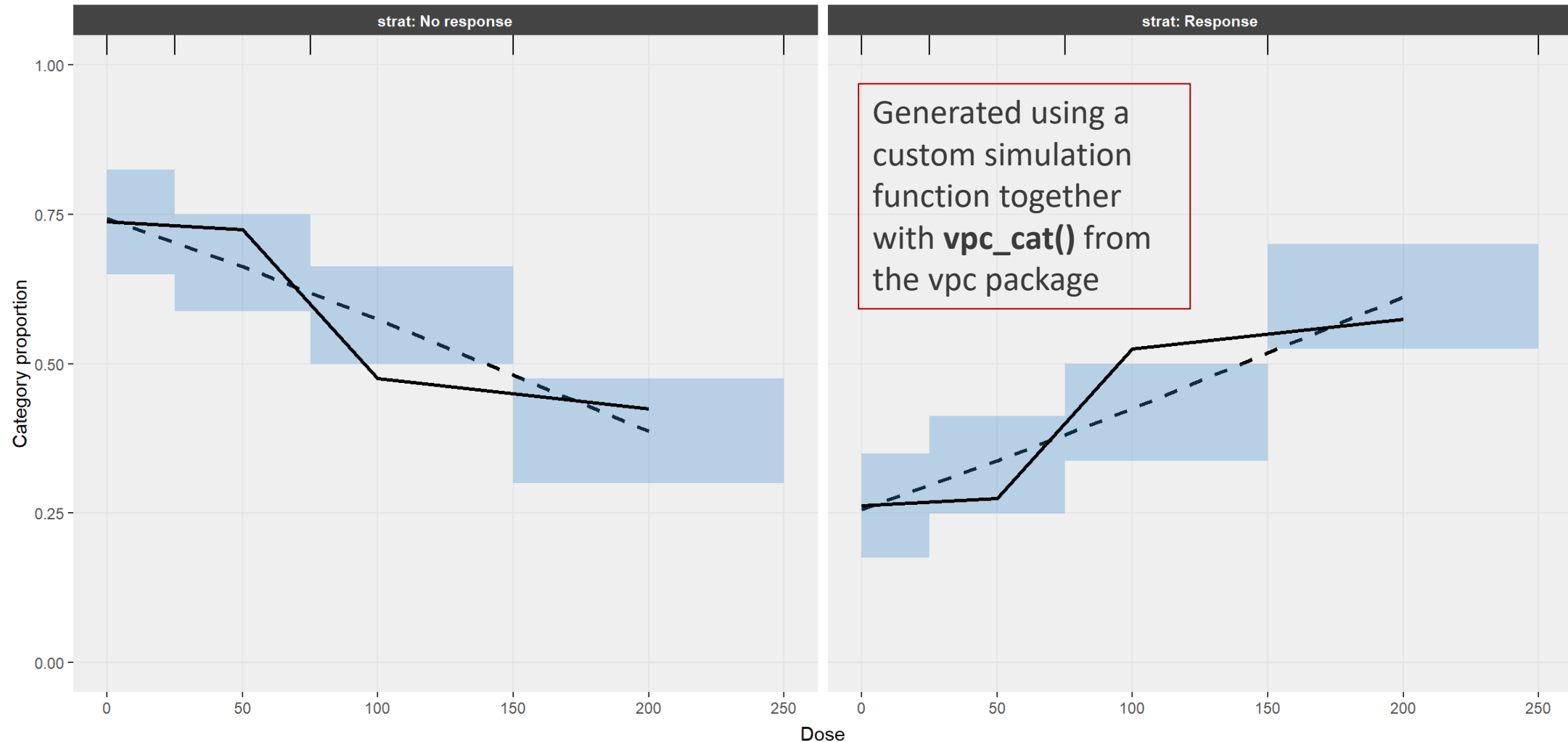
```
cor:beta_dose,logit_p0
```

-0.780

```
Censoring (fit_logistic_binary$censInformation): No censoring
```

A VPC shows our model does the job

Categorical VPC for binary logistic regression



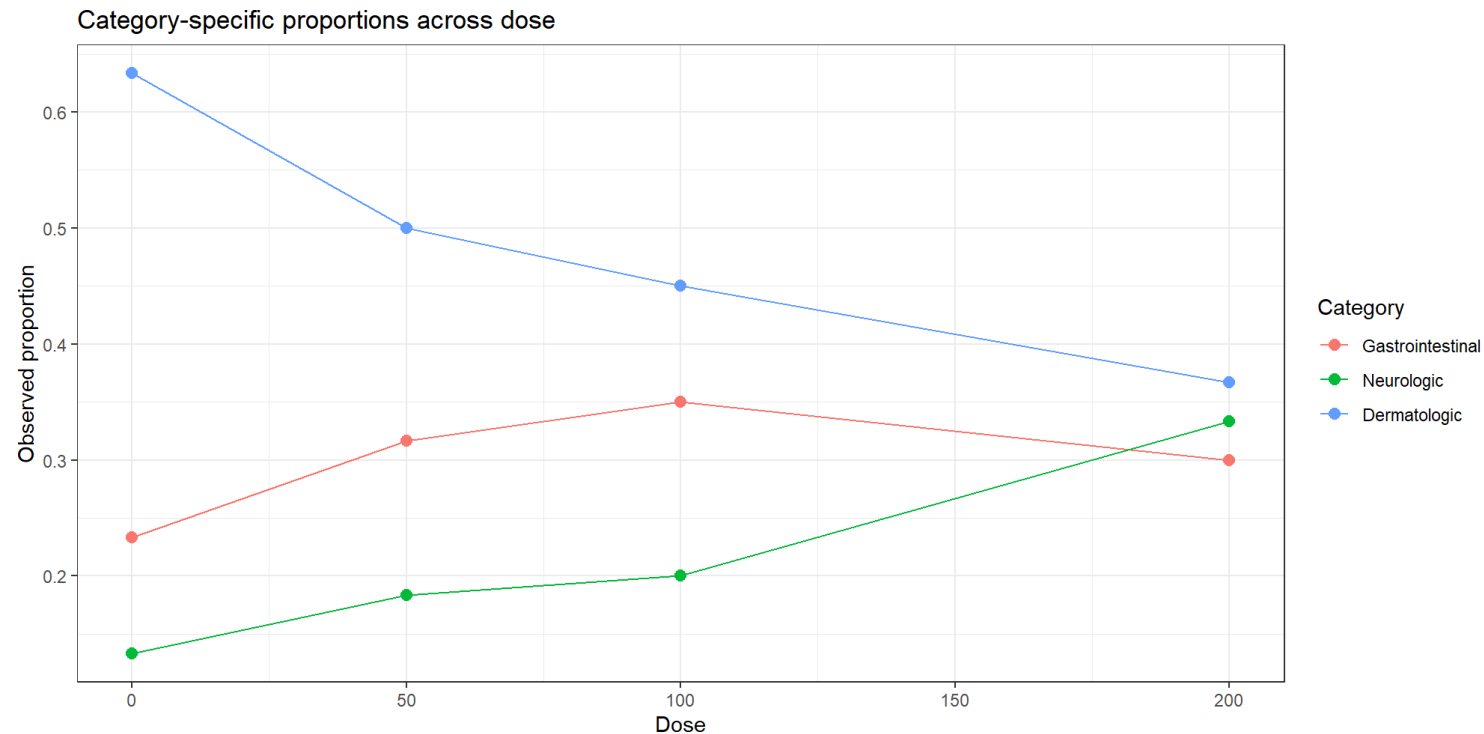
Models for count data

- Counts arise when the endpoint is the number of events in an observation window
 - e.g. seizures per week, infections per treatment cycle, vomiting episodes, device failures
- Count data can have different properties:
 - Equidispersed: for a given individual and covariate pattern, the mean and variance are both the same → **Poisson models**
 - Overdispersed: some subjects have many more events than the population mean predicts, while others have few or none → **negative binomial models**
 - Zero-inflated: disproportionately high proportion of zero counts → **zero-inflated Poisson and negative binomial models**
- **nlmixr2** can handle all of these, but as before, we need to write our own likelihood – see the **Counts** folder for examples

Plan E. Modeling and Simulation of Count Data. CPT: Pharmacometrics & Systems Pharmacology. 2014 Aug 1;3(8):1–12. doi:[10.1038/PSP.2014.27](https://doi.org/10.1038/PSP.2014.27)

Models for unordered categorical data

- Unordered categorical endpoints occur when the outcome is a category, but the categories do not have a natural severity or magnitude ordering
 - e.g. adverse-event type, reason for discontinuation, responder phenotype, etc.
 - Categories are not ranked, just different



Models for unordered categorical data

```
unordered_model <- function() {  
  ini({  
    # Category 2 versus category 1 at DOSE = 100  
    cat2_intercept <- -0.2  
    cat2_dose <- 0.004  
    # Category 3 versus category 1 at DOSE = 100  
    cat3_intercept <- 0.3  
    cat3_dose <- -0.003  
  })  
  model({  
    # Logits for categories 2 and 3 relative to reference category 1  
    lp2 <- cat2_intercept + cat2_dose * (DOSE_MG - 100)  
    lp3 <- cat3_intercept + cat3_dose * (DOSE_MG - 100)  
    # Category 1 is represented by  $\exp(0) = 1$   
    denom <- 1 + exp(lp2) + exp(lp3)  
    p1 <- 1 / denom  
    p2 <- exp(lp2) / denom  
    # Three categories: 1, 2, and the remainder category 3  
    CAT ~ c(p1, p2)  
  })  
}  
# Fit the fixed-effect categorical model  
fit_unordered <- nlmixr(unordered_model, unordered_data, est = "nlm")
```

Models for unordered categorical data

```
> fit_unordered
```

```
— nlmixr2 log-likelihood nlm —
```

	OBJF	AIC	BIC	Log-likelihood	Condition#(Cov)	Condition#(Cor)
lPop	49.78033	498.8708	512.7934	-245.4354	13295.74	3.190193

```
— Time (sec fit_unordered$time): —
```

	setup	optimize	covariance	preprocess	postprocess	table	compress	other
elapsed	0.4020027	0.6104126	4.8e-06	0.07	0	0.02	0	0.3375799

```
— (fit_unordered$parFixed or fit_unordered$parFixedDf): —
```

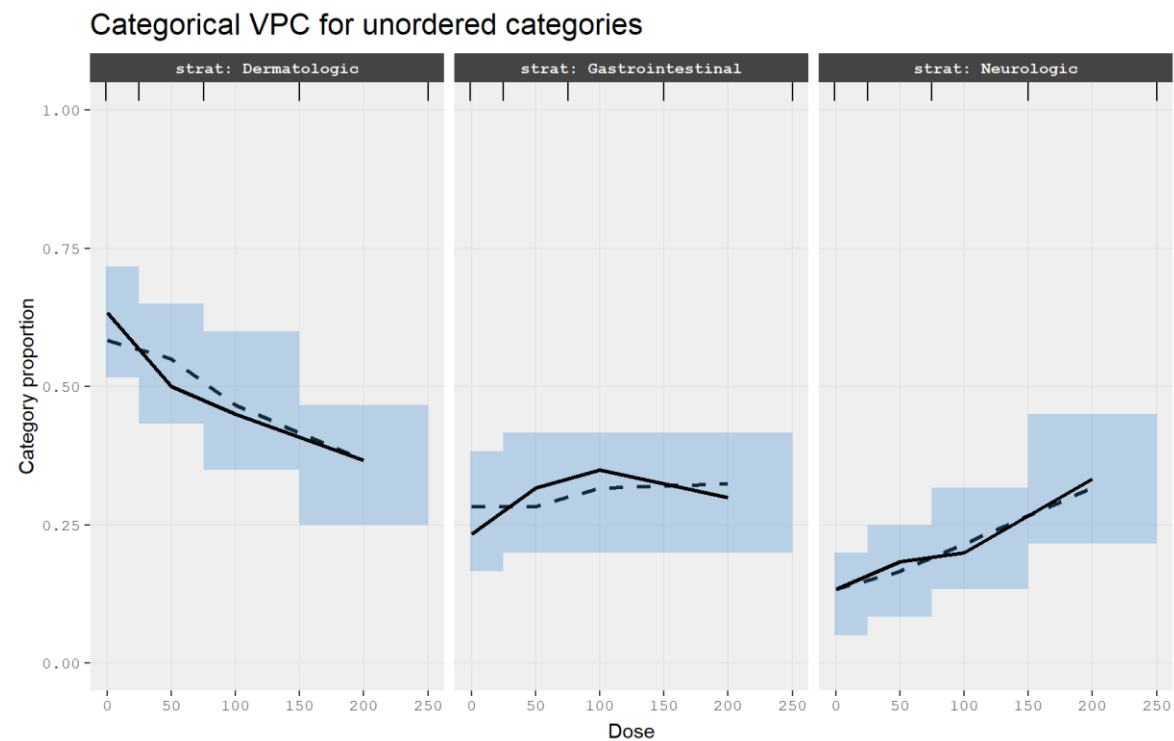
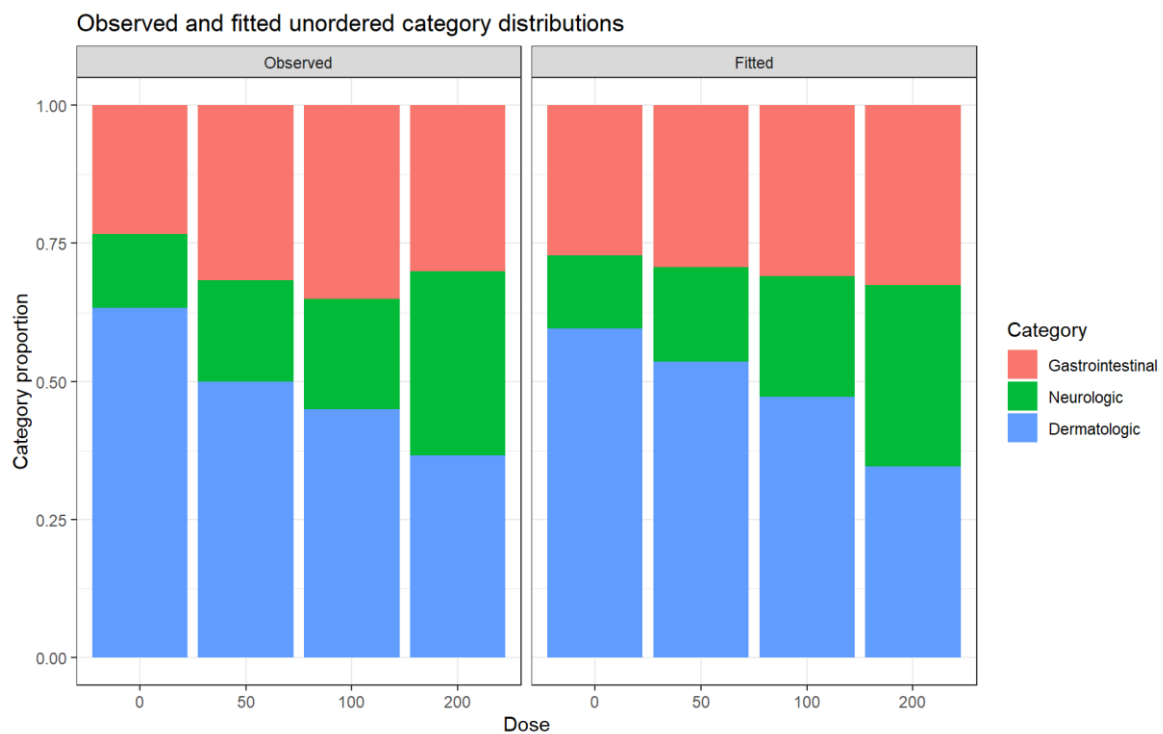
	Est.	SE	%RSE	Back-transformed(95%CI)
cat2_intercept	-0.335	0.147	43.9	-0.335 (-0.624, -0.0470)
cat2_dose	0.00407	0.00212	52.1	0.00407 (-8.67e-5, 0.00823)
cat3_intercept	-0.670	0.173	25.9	-0.670 (-1.01, -0.330)
cat3_dose	0.00873	0.00234	26.9	0.00873 (0.00413, 0.0133)

```
Covariance Type (fit_unordered$covMethod): r (nlm)
```

```
Fixed parameter correlations in fit_unordered$cor
```

```
Censoring (fit_unordered$censInformation): No censoring
```

The model fits well!



Models for ordered categorical data

- Ordered categorical endpoints are common for safety and symptom scores
 - none, mild, moderate, severe; grade 0 through grade 4 toxicity; etc.
- Similar to unordered categories; probabilities must sum to 1...
 - Category 1 = p_1
 - Category 2 = p_2
 - Category 3 = p_3
 - Category 4 = $1 - p_1 - p_2 - p_3$
- The difference is in the cumulative probabilities:
 - $p_1 = cp_1$
 - $p_2 = cp_2 - cp_1$
 - $p_3 = cp_3 - cp_2$
 - $p_4 = 1 - cp_3$
- A convenient way to tackle this is a proportional-odds model with ordered thresholds

Proportional odds model

```
ordinal_model <- function() {  
  ini({  
    # First threshold and positive log-increments to the next thresholds  
    theta1 <- -1  
    log_delta2 <- log(1.4)  
    log_delta3 <- log(1.2)  
    # Dose effect on the latent severity scale  
    beta_dose <- 0.005  
    # Subject-level variability in latent severity  
    eta_sev ~ 0.05  
  })  
  model({  
    # Linear predictor centered at 100 mg for numerical stability  
    lp <- beta_dose * (DOSE - 100) + eta_sev  
    # Ordered thresholds  
    theta2 <- theta1 + exp(log_delta2)  
    theta3 <- theta2 + exp(log_delta3)  
    # Cumulative probabilities  $P(Y \leq k)$   
    cp1 <- expit(theta1 - lp)  
    cp2 <- expit(theta2 - lp)  
    cp3 <- expit(theta3 - lp)  
    # Category probabilities  
    p1 <- cp1  
    p2 <- cp2 - cp1  
    p3 <- cp3 - cp2  
    # Four ordered categories: 1, 2, 3, and the remainder category 4  
    SEV ~ c(p1, p2, p3)  
  })  
}  
# Fit the ordered categorical model  
fit_ordinal <- nlmixr(ordinal_model, ordered_data, est = "laplace")
```

Proportional odds model

```
> fit_ordinal
```

```
— nlmixr2 log-likelihood AGQi (outer: bobyqa; Laplace) —
```

	OBJF	AIC	BIC	Log-likelihood	Condition#(Cov)	Condition#(Cor)
Laplace	256.8937	1002.045	1022.002	-496.0223	28366.89	4.444154

```
— Time (sec fit_ordinal$time): —
```

	setup	optimize	covariance	preprocess	postprocess	table	compress	other
elapsed	0.4422656	0.0925861	0.1551246	0.08	0.01	0.05	0	0.4900237

```
— Population Parameters (fit_ordinal$parFixed or fit_ordinal$parFixedDf): —
```

	Est.	SE	%RSE	Back-transformed(95%CI)	BSV(SD)	Shrink(SD)%
theta1	-0.506	0.140	27.6	-0.506 (-0.780, -0.232)		
log_delta2	0.335	0.0983	29.4	0.335 (0.142, 0.527)		
log_delta3	0.132	0.124	94.1	0.132 (-0.111, 0.376)		
beta_dose	0.00833	0.00170	20.4	0.00833 (0.00500, 0.0117)		
eta_sev					0.764	36.7

It works!

